Apuntes de clase | 2026

Diseño de Compiladores

Prof. Sergio Aranda

Capítulo 1 Clase 1 - Procesamiento de Lenguajes Página 4
1.1 Procesadores de Lenguaje 4
1.1.1 Compilador 4
1.1.2 Traductor 5
1.1.3 Interpretes 5
1.1.4 Procesamiento de la entrada en un Traductor Dirigido por la Sintaxis (TDS) 6
1.1.5 Convertir gramatica predictiva a codigo 9

Capítulo 2 Clase 2 - Traductores dirigidos por sintaxis Página 11
2.1 Ejercicio 1 11
2.1.1 Conversión a una gramática predictiva 11
2.2 Ejercicio 2 - Números romanos 14
2.2.1 Análisis de factores comunes 16
2.3 Ejercicio 3 - Expresiones infijas 18
2.3.1 BNF 19
2.3.2 División de las reglas 19
2.3.3 Acciones semánticas 20

Capítulo 3 Clase 3 - Traductores dirigidos por sintaxis Página 21
3.1 Ejercicio 1 - Declaraciones de arreglos 21
3.1.1 Árbol sintáctico de prueba 22
3.1.2 Código fuente del analizador sintáctico 22
3.2 Ejercicio 2 - Multiplicación de enteros de 0 a 9 23
3.2.1 Árbol sintáctico del ejemplo 23
3.2.2 Código fuente del analizador sintáctico 23
3.3 Ejercicio 3 - TDS de expresiones 24
3.3.1 Reglas semánticas 24
3.3.2 Árbol sintáctico de prueba 25
3.3.3 Eliminación de la recursión por la izquierda 25
3.3.4 Conversión de la gramática de expresiones 25
3.3.5 Conversión a codigo fuente 26
3.4 Ejercicio 4 - Evaluación de expresiones con multiplicación 26
3.4.1 Conversión a gramática predictiva 27

Capítulo 4 Clase 4 - Compiladores Página 28
4.1 Visión general del compilador 28

4.2 Análisis léxico 28
4.2.1 Lexemas, tokens y patrones 28
4.2.2 Expresiones regulares 29
4.3 Análisis sintáctico 30
4.4 Relación entre las etapas 31
4.5 Ejemplo de gramática y definiciones regulares 31
4.5.1 Diagramas de estados 32
4.5.2 Autómatas finitos no deterministas 34
4.5.3 Construcción de Thompson 34
4.5.4 AFN de las definiciones regulares 35
4.5.5 Algoritmo de Cerradura-ε (Algoritmo de Subconjuntos) 36
4.5.6 Resumen y Aclaraciones 40

Capítulo 5 Ejercicios 1 - BNF Página 41

Capítulo 6 Ejercicio 2 - TDS Página 42
6.0.1 Solución con la gramática propuesta 42
6.0.2 Conversión a una gramática predictiva 43
6.0.3 Pseudocódigo del analizador sintáctico 44

1

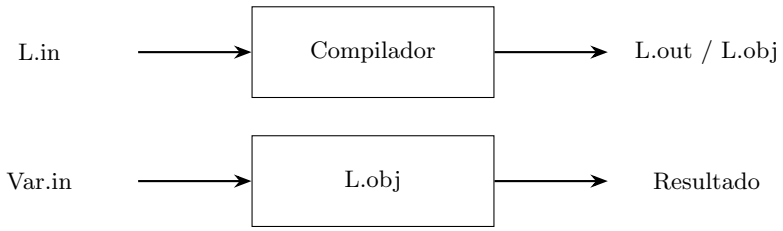
Clase 1 - Procesamiento de Lenguajes

1.1 Procesadores de Lenguaje

Son herramientas que nos permiten introducir un lenguaje de entrada y generar otra salida, que puede ser otro lenguaje o un conjunto de instrucciones para una computadora.

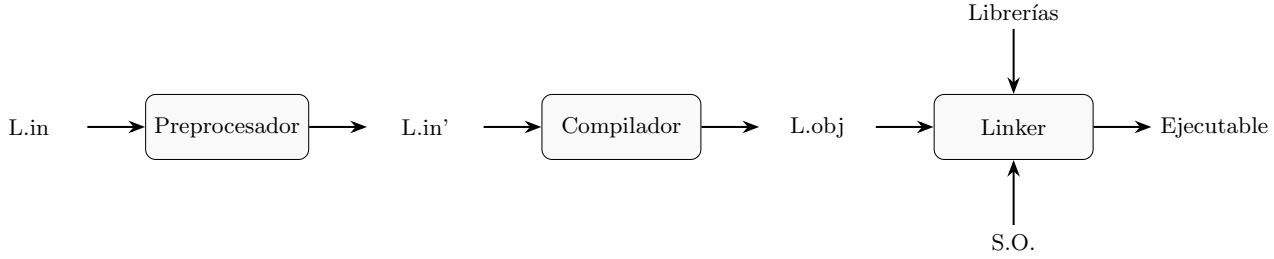
1.1.1 Compilador

Programa que traduce un lenguaje de entrada a un lenguaje de salida, que generalmente es un lenguaje de máquina o lenguaje objeto. Asi podemos decir que un compilador es un traductor de alto nivel a bajo nivel. El programa en lenguaje objeto puede ser ejecutado directamente por la computadora, transformando variables de entrada en variables de salida (resultado).



Caracteristicas

- Portabilidad de ejecutable.
- Tiempo de ejecución.
- Ocupa menos memoria.



Pre-analisis

Eliminamos todo lo que no es necesario para el analisis del compilador, como espacios en blanco, comentarios, etc. Esto se hace con un preprocesador.

Análisis

El análisis se centra en el lenguaje de entrada, el cual debe estar bien definido. Se divide en tres partes:

- **Léxico:** Se encarga de generar, a partir del lenguaje de entrada, los lexemas, que son las unidades básicas del lenguaje, como palabras clave, identificadores, operadores, etc. El lexer carga estos elementos como tokens en una tabla de símbolos, que almacena los lexemas y sus atributos, como el tipo de dato, el valor, etc.
- **Sintáctico:** Se encarga de analizar la estructura del lenguaje de entrada a partir de los tokens generados por el analizador léxico y generar un árbol de derivación, que es una representación gráfica de la estructura del lenguaje. Verifica que se cumplan las reglas gramaticales del lenguaje de entrada.
- **Semántico:** Se encarga de analizar el significado del lenguaje de entrada y generar un conjunto de instrucciones para la computadora, que son las instrucciones que se ejecutarán en el lenguaje objeto. Verifica que se cumplan las reglas semánticas del lenguaje de entrada, como el tipo de dato, el valor, etc.

> **Nota 1.1.1** Gestor de Errores:
>
> Se encarga de manejar los errores que se generan durante el análisis, como errores léxicos, sintácticos y semánticos. Se puede detener el análisis y mostrar un mensaje de error, o continuar con el análisis.

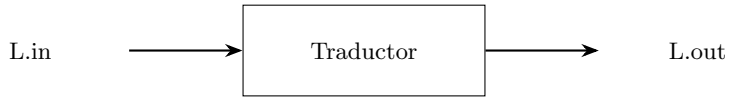
Síntesis

El proceso de síntesis se encarga de transformar la representación intermedia en el lenguaje objeto. Se divide en:

- **Optimización:** Se encarga de mejorar el rendimiento del lenguaje objeto, eliminando instrucciones innecesarias y mejorando la eficiencia del código.
- **Construcción:** Se encarga de generar el lenguaje objeto a partir del árbol de derivación y las instrucciones generadas por el análisis semántico.

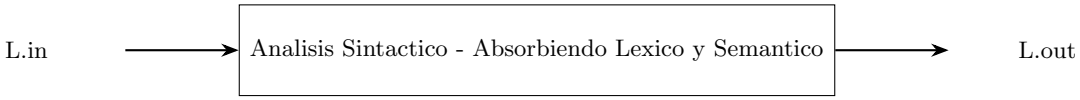
1.1.2 Traductor

Programa que traduce un lenguaje de entrada a un lenguaje de salida, con lenguajes con niveles de complejidad equiparados.



Traduccion Dirigida por la Sintaxis (TDS)

La traducción dirigida por la sintaxis traduce el lenguaje de entrada a uno de salida agrupando la parte de analisis lexico, sintactico y semantico en un solo paso, generando el lenguaje de salida directamente. Tiene dos partes: una parte en la que procesamos la entrada, y una parte donde construimos la salida.



1.1.3 Interpretes

Programa que recibe un lenguaje de entrada y las variables entrada (datos) para generar directamente un resultado (salida), sin generar un lenguaje objeto. El resultado son las instrucciones que se ejecutan directamente en la maquina.



1.1.4 Procesamiento de la entrada en un Traductor Dirigido por la Sintaxis (TDS)

Definición 1.1.1 BNF: Backus-Naur Form

Es una jerarquía de reglas de producción que definen la sintaxis de un lenguaje de programación. Un sistema de producciones genera un lenguaje.

Definición 1.1.2 ϵ

Es la cadena vacía, que representa la ausencia de un símbolo. Causa la extinción de una producción.

Definición 1.1.3 Σ

Es el alfabeto del lenguaje, que es el conjunto finito de símbolos que se pueden utilizar para formar cadenas. Ejemplos de alfabetos son $\{0, 1\}$, el conjunto de letras, el conjunto de dígitos y el código ASCII.

Definición 1.1.4 Cadena

Una cadena sobre un alfabeto es una secuencia finita de símbolos obtenidos a partir de ese alfabeto. La longitud de una cadena está determinada por la cantidad de símbolos que la forman. Por ejemplo, la cadena `universidad` tiene longitud 11.

Definición 1.1.5 Lenguaje

Es cualquier conjunto contable de cadenas definidas sobre un alfabeto. Los lenguajes pueden definirse por extensión o por comprensión.

Ejemplo 1.1.1 BNF con a y b como terminales

$A \to A\,a\,B\,b \mid \epsilon$
$B \to b \mid \epsilon$
$\Sigma = \{a, b\}$
A y B son variables sintacticas.

Ejemplo 1.1.2 BNF con A y B como terminales

$S \to ASB$
$\Sigma = \{A, B\}$
S es una variable sintactica.

Ejemplo 1.1.3 Definición de Gramatica Libre de Contexto (CFG):

$cad \to cad + cad \mid cad - cad \mid digito$
$digito \to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$
$\Sigma = \{+, -, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Arbol Sintactico:

Es una representación gráfica de la estructura sintáctica de una cadena de texto, donde cada nodo representa un símbolo de la gramática y las ramas representan las reglas de producción.

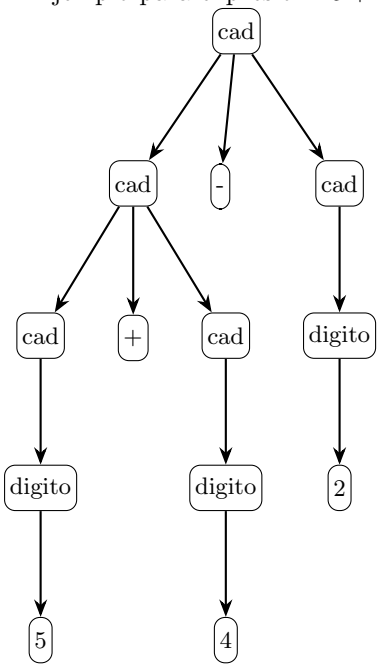
Derivacion:

Es el proceso de sustituir los símbolos no terminales por sus correspondientes producciones hasta obtener una cadena de símbolos terminales.

El padre en un arbol sintáctico es el símbolo no terminal que se está expandiendo, y los hijos son los símbolos que se están generando a partir de esa expansión. Por ejemplo en el arbol (A con hijos x, y, z) se tiene la derivación A $\rightarrow$ x y z. La raiz y nodos intermedios son variables sintácticas, mientras que las hojas son símbolos terminales.

Hay dos formas de validar que una cadena sea válida en una gramática, top-down y bottom-up. En top-down se empieza desde la raiz y se va expandiendo hasta llegar a las hojas, mientras que en bottom-up se empieza desde las hojas y se va reduciendo hasta llegar a la raiz.

Ejemplo para expresion "$5 + 4 - 2$":



Se puede avanzar en la entrada de izquierda a derecha como tambien se puede avanzar leyendo de derecha a izquierda.

Si tomamos un camino incorrecto al verificar una entrada puede ocurrir backtracking, retocediendo para probar nuevamente. Si la gramatica es muy compleja puede ocurrir un backtracking infinito, por lo que se debe tener cuidado al diseñar la gramatica. Si hay dos maneras de llegar a un mismo resultado, se dice que la gramatica es ambigua. Por ejemplo, la expresion 5 + 4 - 2 puede ser interpretada como (5 + 4) - 2 o como 5 + (4 - 2), lo que genera dos arboles sintacticos diferentes.

Pregunta 1.1.1 Ejercicios

¿Son estas gramáticas ambiguas?

a) $S \rightarrow 0S1 \mid 01$
$\Sigma = \{0, 1\}$

b) $S \rightarrow S(S)S \mid \epsilon$
$\Sigma = \{(,)\}$

c) $S \rightarrow aSbS \mid bSaS \mid \epsilon$
$\Sigma = \{a, b\}$

Posibles Contrajemplos (cadenas ambiguas):

- a) `0011` $\rightarrow$ No es ambiguo, solo se puede derivar por el medio y se forma un arbol infinito. (Pero con otro ejemplo es ambiguo por tener factor comun por la izquierda)

- b) `()()` → Es ambiguo porque puede ser derivado de dos maneras distintas, por izuqierda y por derecha.
- c) `abab` → Es ambiguo porque puede ser derivado de dos maneras distintas, por izuqierda y por derecha.

Definición 1.1.6 Gramatica Predictiva

Una gramatica es predictiva si y solo si no tiene recursión a la izquierda, que no tiene factor comun por la izquierda, y que no tiene factor comun indirecto por la izquierda (no es ambigua). Esto permite que se pueda predecir la producción a utilizar a partir del símbolo no terminal y el símbolo terminal que se está leyendo.

Ejemplo eliminando recursión a la izquierda:

- $A \to A\alpha \mid \beta$
  $\equiv$
- $A \to \beta R$
- $R \to \alpha R \mid \epsilon$

Factor comun por la izquierda:

- $A \to Ba \mid Bb$
  $\equiv$
- $A \to BR$
- $R \to a \mid b$

Factor comun indirecto por la izquierda:

- $A \to Ba \mid Cd$
- $B \to ba \mid \epsilon$
- $C \to bd \mid \epsilon$

Definición 1.1.7 Conjunto Primero

El conjunto *Primero* de un símbolo no terminal es el conjunto de símbolos terminales que pueden aparecer al comienzo de alguna cadena derivada a partir de dicho símbolo. Si un símbolo puede derivar ϵ, entonces para calcular el conjunto Primero de una cadena se continúa evaluando el siguiente símbolo de la cadena. Si toda la cadena puede derivar ϵ, entonces ϵ pertenece al conjunto Primero.

$$\begin{aligned} Primero(a) &= \{a\} \\ Primero(A) &= \{\text{terminales que pueden iniciar una derivación de } A\} \\ Primero(ABC) &= \cdots \end{aligned}$$

Afirmación 1.1.1 Detección de factor común indirecto mediante el conjunto Primero

Sea una producción

$$A \to \alpha_1 \mid \alpha_2 \mid \cdots \mid \alpha_n.$$

Para determinar si existe factor común por la izquierda indirecto, se calcula el conjunto *Primero* de cada alternativa α_i. Si existen dos alternativas tales que

$$Primero(\alpha_i) \cap Primero(\alpha_j) \neq \varnothing,$$

entonces ambas pueden comenzar con el mismo símbolo terminal y existe un conflicto que indica la presencia de un posible factor común por la izquierda (directo o indirecto). En ese caso, la gramática debe transformarse mediante factorización.

Si, por el contrario,

$$Primero(\alpha_i) \cap Primero(\alpha_j) = \varnothing \qquad \forall\, i \neq j,$$

entonces las alternativas son distinguibles por el primer símbolo de entrada y no existe factor común por la izquierda.

Ejemplo 1.1.4 Eliminando factor común indirecto por la izquierda

La gramática inicial es:

$$\begin{aligned} cad &\to cad + cad \mid cad - cad \mid dig \\ dig &\to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

1. **Eliminar la recursión por la izquierda.**

$$\begin{aligned} cad &\to dig\, R \\ R &\to + cad\, R \mid - cad\, R \mid \epsilon \\ dig &\to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

2. **Eliminar el factor común por la izquierda.**

   No posee factor común.

3. **Eliminar el factor común indirecto por la izquierda.**

   Para ello se utiliza el conjunto *Primero*.

$$\begin{aligned} P(cad) &= \{0,1,2,3,4,5,6,7,8,9\} \\ P(R) &= P(+cadR) \cup P(-cadR) \cup P(\epsilon) = \{+,-,\epsilon\} \\ P(dig) &= \{0\} \cup \{1\} \cup \{2\} \cup \{3\} \cup \{4\} \cup \{5\} \cup \{6\} \cup \{7\} \cup \{8\} \cup \{9\} \\ &= \{0,1,2,3,4,5,6,7,8,9\} \end{aligned}$$

   Como

$$\begin{aligned} Primero(+cadR) &= \{+\}, \\ Primero(-cadR) &= \{-\}, \\ Primero(\epsilon) &= \{\epsilon\}, \end{aligned}$$

   se verifica que

$$\{+\} \cap \{-\} = \{+\} \cap \{\epsilon\} = \{-\} \cap \{\epsilon\} = \varnothing.$$

   Por el criterio anterior, las alternativas de R son distinguibles por su primer símbolo terminal y, por lo tanto, la gramática no posee factor común indirecto por la izquierda.

1.1.5 Convertir gramatica predictiva a codigo

En el ejemplo anterior, se puede generar un pseudocodigo para el analizador sintactico, que se puede convertir a cualquier lenguaje de programacion de forma sencilla.

```
cod() {
    dig();R();
}

R(){
    if (token == '+') {
```

```
        match('+');
        cod();
        R();
    } else if (token == '-') {
        match('-');
        cod();
        R();
    } else; // Caso de cadena vacia
}

dig() {
    if (token == '0') match('0');
    else if (token == '1') match('1');
    else if (token == '2') match('2');
    else if (token == '3') match('3');
    else if (token == '4') match('4');
    else if (token == '5') match('5');
    else if (token == '6') match('6');
    else if (token == '7') match('7');
    else if (token == '8') match('8');
    else if (token == '9') match('9');
}

match(t) {
    if (input == t) {
        input = getnextinput();
    } else {
        error();
    }
}
```

Nota 1.1.2 Acerca de Manejo de Errores en Codigo

Realizamos captura de error en match(), este es nuestro manejador de errores por defecto. Realizamos if (input == t) para verificar que el token actual es el esperado, si no lo es, se llama a error() para manejar el error. Es fundamental mantener un else para capturar cualquier token inesperado.

2

Clase 2 - Traductores dirigidos por sintaxis

2.1 Ejercicio 1

Consejo 2.1.1 Consejo del profesor

Al querer convertir un lenguaje a BNF, hay que enfocarse primero en describir el lenguaje, sin dejar que ninguna cadena válida quede fuera de la gramática. Luego, se convierte la gramática en una gramática predictiva utilizando las técnicas estudiadas. Por ejemplo, se puede describir un lenguaje de listas de números enteros positivos separados por comas (una lista unitaria también se admite).

2.1.1 Conversión a una gramática predictiva

1. $lista \to lista, nro \mid nro$ (recursión por la izquierda).
   Se convierte en:
   $$\begin{aligned} lista &\to nroR, \\ R &\to , nroR \mid \varepsilon. \end{aligned}$$
2. $lista \to nro, lista \mid nro$ (crece por la derecha, pero todavía tiene factor común).
   Se convierte en:
   $$\begin{aligned} lista &\to nroR, \\ R &\to , lista \mid \varepsilon. \end{aligned}$$
3. $nro \to nro\ dig \mid dig$ (recursión por la izquierda).
   Se convierte en:
   $$\begin{aligned} nro &\to digR, \\ R &\to digR \mid \varepsilon. \end{aligned}$$
4. $dig \to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$ (no tiene recursión ni factor común).

Finalmente:

$$\begin{aligned} lista &\to nroR, \\ R &\to , nroR \mid \varepsilon, \\ nro &\to digR', \\ R' &\to digR' \mid \varepsilon, \\ dig &\to 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9. \end{aligned}$$

Conjunto Primero

$$
\begin{aligned}
P(lista) &= \{0,1,2,3,4,5,6,7,8,9\},\\
P(R) &= \{\ ,,\varepsilon\ \},\\
P(nro) &= \{0,1,2,3,4,5,6,7,8,9\},\\
P(R') &= \{0,1,2,3,4,5,6,7,8,9,\varepsilon\},\\
P(dig) &= \{0,1,2,3,4,5,6,7,8,9\}.
\end{aligned}
$$

Código del analizador sintáctico

```
R() {
    if (input == ',') {
        match(',');
        nro();
        R();
    } else; // Caso de cadena vacía
}
```

Para generar un lenguaje de salida usamos reglas o acciones semánticas.

Para convertir la gramática en una gramática más simple antes de definir las reglas semánticas, la dividimos con la técnica de divide y vencerás.

Quiero poder contar cuántos números hay en la lista, cuántos son pares y cuántos son impares (tres variables: cantidad, pares e impares).

Por ejemplo, la gramática anterior quedará de la siguiente manera:

Cuadro 2.1: Reglas semánticas para el conteo de listas

BNF	**Reglas semánticas**
lista → nroR	Primero nro y luego R: if atoi($nro.n$) %2 = 0 then $lista.p = 1 + R.p$; else $lista.i = 1 + R.i$; `printf(lista.c);` $lista.c = 1 + R.c$; `printf(lista.p,lista.i)`
R →, nroR	R.c = 1 + R.c; if atoi(nro.n) %2 = 0 : R.p = 1 + R.p; else : R.i = 1 + R.i
R → ε	R.c = 0; R.p = 0; R.i = 0
nro → digR′	nro.n = dig.x || R′.x
R′ → digR′	R′.x = dig.x || R′.x
R′ → ε	R′.x = NULL
dig → 0	dig.x = '0'
dig → 1	dig.x = '1'
dig → 2	dig.x = '2'
dig → 3	dig.x = '3'
dig → 4	dig.x = '4'
dig → 5	dig.x = '5'
dig → 6	dig.x = '6'
dig → 7	dig.x = '7'
dig → 8	dig.x = '8'
dig → 9	dig.x = '9'

Donde x es el valor del número, c es la cantidad de números en la lista, p es la cantidad de números pares e i es la cantidad de números impares.

La limitación de la definición de reglas semánticas en los TDS es que solo existen variables semánticas. Las variables pueden ser sintetizadas (cuando su valor solo depende del valor de sus hijos) o heredadas (cuando su valor depende de sus hermanos).

Como ayuda, podemos analizar el árbol del TDS; por ejemplo, analizando la cadena $50, 8, 1$ y observando qué valores de las variables semánticas esperamos.

Código de salida:

```
char dig() {
    if (input == '0') {
        match('0');
        return ('0');
    }
    else if (input == '1') {
        match('1');
        return ('1');
    }
    ...
    else if (input == '9') {
        match('9');
        return ('9');
    }
}

R'() {
    if (input == '0' || input == '1' || input == '2' || input == '3' ||
        input == '4' || input == '5' || input == '6' || input == '7' ||
        input == '8' || input == '9') {
        return (dig() || R'());
    } else {
        return '\0';
    }
}

nro() {
    return (dig() || R'());
}

R() {
    int c, i, p;
    if (input == ',') {
        match(',');
        nro();
        R();
    }
    else {
        c = 0; p = 0; i = 0;
    }
}

lista() {
    int c, p, i;
    if (nro() % 2 == 0)
        p = 1; c = 1; i = 0;
    else
        i = 1; c = 1; p = 0;
    R = R(); // Asumimos que se define un struct R con variables c, p, i
    c = c + R.c; p = p + R.p; i = i + R.i;

    // El print en realidad debería ir en el main()
    printf("Cantidad: %d, Pares: %d, Impares: %d\n", lista.c, lista.p, lista.i);
}
```

Nota 2.1.1 Observación

Las variables semánticas son, en realidad, valores de retorno de las funciones que representan a los no terminales. Por ejemplo, la función *lista*() retorna un `struct` con los valores de c, p e i. Se debe dejar en evidencia el manejo de valores de retorno en el código (pseudo-C) y mantener el orden de llamada de las funciones correspondientes a las reglas del BNF.

2.2 Ejercicio 2 - Números romanos

Pregunta 2.2.1 Ejercicio 2 - Números romanos

Escribir un TDS que acepte un lenguaje formado por números romanos del 1 al 10 y retorne el número correspondiente.

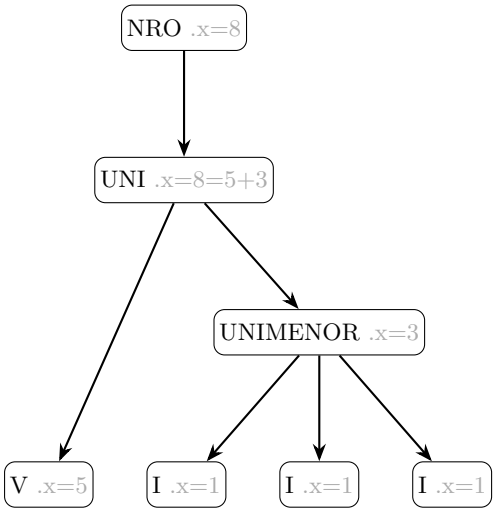
Ejemplo 2.2.1 Ejemplo

$VIII \longrightarrow$ [TDS] $\longrightarrow 8$.

BNF propuesto

- $NRO \rightarrow DEC \mid UNI$
- $DEC \rightarrow X \mid \varepsilon$
- $UNI \rightarrow UNIMENOR \mid IV \mid VUNIMENOR \mid IX$
- $UNIMENOR \rightarrow I \mid II \mid III \mid \varepsilon$

Primer paso: árbol sintáctico



Divide y vencerás: dividir las reglas semánticas

Cuadro 2.2: Paso 1: reglas semánticas de la gramática propuesta

BNF	**Reglas semánticas**
NRO → DEC	NRO.x = DEC.x; `printf(NRO.x)`
NRO → UNI	NRO.x = UNI.x; `printf(NRO.x)`
DEC → X	DEC.x = 10
DEC → ε	DEC.x = 0
UNI → UNIMENOR	UNI.x = UNIMENOR.x
UNI → IV	UNI.x = 4
UNI → V UNIMENOR	UNI.x = 5 + UNIMENOR.x
UNI → IX	UNI.x = 9
UNIMENOR → I	UNIMENOR.x = 1
UNIMENOR → II	UNIMENOR.x = 2
UNIMENOR → III	UNIMENOR.x = 3
UNIMENOR → ε	UNIMENOR.x = 0

Paso dos: verificar que la gramática sea predictiva

- Recursión por la izquierda: no tiene.
- Factor común directo: tiene; hay que resolverlo.

Resolución del factor común directo:

$$\begin{aligned} UNI &\to UNIMENOR \mid IR \mid V\,UNIMENOR, \\ R &\to V \mid X, \\ UNIMENOR &\to IS \mid \varepsilon, \\ S &\to IS' \mid \varepsilon, \\ S' &\to I \mid \varepsilon. \end{aligned}$$

Reglas semánticas resultantes:

Cuadro 2.3: Paso 2: resolución del factor común directo

BNF	**Reglas semánticas**
NRO → DEC	NRO.x = DEC.x; `printf(NRO.x)`
NRO → UNI	NRO.x = UNI.x; `printf(NRO.x)`
DEC → X	DEC.x = 10
DEC → ε	DEC.x = 0
UNI → UNIMENOR	UNI.x = UNIMENOR.x
UNI → IR	UNI.x = R.x − 1
R → V	R.x = 5
R → X	R.x = 10
UNI → V UNIMENOR	UNI.x = 5 + UNIMENOR.x
UNIMENOR → IS	UNIMENOR.x = 1 + S.x
UNIMENOR → ε	UNIMENOR.x = 0
S → I	S.x = 1 + S′.x
S → ε	S.x = 0
S′ → I	S′.x = 1
S′ → ε	S′.x = 0

Nota 2.2.1 Nota

Es recomendable y conveniente probar el árbol sintáctico con la gramática final para verificar que se obtienen los mismos resultados que con la gramática inicial, utilizando cadenas de entrada que sigan las derivaciones de las reglas modificadas.

2.2.1 Análisis de factores comunes

Factor común indirecto:

Conjunto Primero:

$$\begin{aligned} P(NRO) &= \{I, \varepsilon\} \cup \{X, \varepsilon\} = \{I, V, X\}, \quad \text{hay factor común indirecto,} \\ P(DEC) &= \{X, \varepsilon\}, \\ P(UNI) &= \{I\} \cup \{V\} \cup \{I, \varepsilon\} = \{I, V\}, \quad \text{hay factor común indirecto,} \\ P(R) &= \{V, X\}, \\ P(UNIMENOR) &= \{I, \varepsilon\}, \\ P(S) &= \{I, \varepsilon\}, \\ P(S') &= \{I, \varepsilon\}. \end{aligned}$$

El factor común indirecto de NRO se puede solucionar eliminando el ε de la regla DEC, ya que está más arriba en la jerarquía.

Nota 2.2.2 Tip

Cuando se debe modificar el vacío (ε), modificar las reglas más altas en la jerarquía suele ser más sencillo.

Para resolver el factor común indirecto en UNI, pasamos a un caso de factor común directo, ya que UNI tiene dos reglas que comienzan con I, y una de ellas es $UNIMENOR$, que a su vez tiene una regla que comienza con I. Por lo tanto, podemos reescribir UNI como:

$$\begin{aligned} UNI &\to UNIMENOR \mid V\,UNIMENOR, \\ UNIMENOR &\to IR \mid IS \mid \varepsilon. \end{aligned}$$

Solucionamos:

$$\begin{aligned} UNIMENOR &\to IZ \mid \varepsilon, \\ Z &\to R \mid S. \end{aligned}$$

Ahora modificamos las reglas semánticas:

Cuadro 2.4: Paso 3: eliminación del factor común indirecto

BNF	**Reglas semánticas**
NRO → DEC	NRO.x = DEC.x; `printf(NRO.x)`
NRO → UNI	NRO.x = UNI.x; `printf(NRO.x)`
DEC → X	DEC.x = 10
DEC → ε	Eliminada
UNI → UNIMENOR	UNI.x = UNIMENOR.x
UNI → IR	Eliminada
R → V	R.x = 3
R → X	R.x = 8
UNI → V UNIMENOR	UNI.x = 5 + UNIMENOR.x
UNIMENOR → IZ	UNIMENOR.x = I.x + Z.x
UNIMENOR → ε	UNIMENOR.x = 0
Z → R	Z.x = R.x
Z → S	Z.x = S.x
S → I	S.x = 1 + S′.x
S → ε	S.x = 0
S′ → I	S′.x = 1
S′ → ε	S′.x = 0

Convertimos la gramática en predictiva, pero hay una observación importante: al hacer una modificación en la gramática no formulada matemáticamente, debemos comprobar que no se hayan introducido nuevas cadenas válidas en el lenguaje (basura). Por ejemplo, en este caso agregamos la cadena VIV como válida, la cual no debe ser validada.

En vez de eso, retrocedemos al paso anterior y probamos reescribir UNI de la siguiente manera:

$$UNI \rightarrow IS \mid \varepsilon \mid IR \mid V\,UNIMENOR.$$

Resolviendo el factor común:

$$UNI \rightarrow IW \mid \varepsilon \mid V\,UNIMENOR,$$
$$W \rightarrow S \mid R.$$

Conjunto Primero:

$$P(UNI) = \{I, V, \varepsilon\}, \qquad P(W) = \{I, V\}.$$

Se resolvió el factor común indirecto.

Ahora reescribimos las reglas semánticas:

Cuadro 2.5: Paso 4: reglas semánticas finales

BNF	**Reglas semánticas**
NRO → DEC	NRO.x = DEC.x; `printf(NRO.x)`
NRO → UNI	NRO.x = UNI.x; `printf(NRO.x)`
DEC → X	DEC.x = 10
UNI → IW	UNI.x = 1 + W.x
W → S	W.x = S.x
W → R	W.x = R.x
R → V	R.x = 3
R → X	R.x = 8
UNI → V UNIMENOR	UNI.x = 5 + UNIMENOR.x
UNI → ε	UNI.x = 0
UNIMENOR → IZ	Eliminada
UNIMENOR → IS	UNIMENOR.x = 1 + Z.x
UNIMENOR → ε	UNIMENOR.x = 0
S → I	S.x = 1 + S′.x
S → ε	S.x = 0
S′ → I	S′.x = 1
S′ → ε	S′.x = 0

Finalmente, podemos escribir el pseudocódigo para el analizador sintáctico directamente utilizando la gramática predictiva final y las reglas semánticas finales.

Código:

```
UNI() {
    if (input == 'I') {
        match('I');
        return (1 + W());
    }
    else if (input == 'V') {
        match('V');
        return (5 + UNIMENOR());
    }
    else
        return 0;
}

W() {
    if (input == 'V' || input == 'X')
        return (R());
    else
        return S();
}

...
```

2.3 Ejercicio 3 - Expresiones infijas

De una expresión infija, por ejemplo $3 + 5$, generar un TDS que retorne la expresión en notación postfija, por ejemplo 3 5 +.

2.3.1 BNF

- $expr \rightarrow expr + term \mid expr - term \mid term$
- $term \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

2.3.2 División de las reglas

1) $expr \rightarrow term + term$
2) $expr \rightarrow term - term$
3) $expr \rightarrow term$
4) $term \rightarrow 0$
5) $term \rightarrow 1$
6) ...
7) $term \rightarrow 9$

Cuadro 2.6: Expresiones infijas: reglas semánticas iniciales

BNF	**Reglas semánticas**
expr → expr + term	expr.x = expr.x || term.x ||′ +′
expr → expr − term	expr.x = expr.x || term.x ||′ −′
expr → term	expr.x = term.x
term → 0	term.x = '0'
term → 1	term.x = '1'
...	...
term → 9	term.x = '9'

Esto no sirve para construir pseudocódigo, ya que no es un TDS predictivo porque tiene recursión por la izquierda. Por lo tanto, debemos eliminar la recursión por la izquierda.

Para eliminarla:

- $expr \rightarrow termR$
- $R \rightarrow +termR \mid -termR \mid \varepsilon$
- $term \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Reescribiendo las reglas:

Cuadro 2.7: Expresiones infijas: reglas semánticas después de eliminar la recursión

BNF	**Reglas semánticas**
expr → termR	expr.x = term.x || R.x
R → + termR	R.x = term.x ||′ +′ || R.x
R → − termR	R.x = term.x ||′ −′ || R.x
R → ε	R.x = '\0'
term → 0	term.x = '0'
term → 1	term.x = '1'
...	...
term → 9	term.x = '9'

Probar el árbol de $3 + 4 - 2$, que debe retornar 3 4 + 2 −.

2.3.3 Acciones semánticas

El libro plantea que las reglas semánticas funcionan fácilmente en el 90 % de los casos, pero en el 10 % restante son difíciles de utilizar. Para este último grupo de casos podemos reemplazar las reglas semánticas por **acciones semánticas** (en el libro: *esquemas de traducción*), que son instrucciones de código que se ejecutan en el momento de la derivación de la regla y no al final como las reglas semánticas. Por ejemplo, para el caso de expresiones infijas podemos utilizar acciones semánticas para generar la notación postfija.

- $expr \to termR$
- $R \to +\, term$ {`printf('+')`} R
- $R \to -\, term$ {`printf('-')`} R
- $R \to \varepsilon$
- $term \to 0$ {`printf('0')`}

Observamos cómo las acciones semánticas pasan a ser nodos del árbol sintáctico.

Nota 2.3.1 Sobre las reglas y acciones semánticas en el libro

El libro plantea la teoría y las propiedades en base a acciones semánticas, pero estas reglas son igualmente válidas para las reglas semánticas.

3

Clase 3 - Traductores dirigidos por sintaxis

3.1 Ejercicio 1 - Declaraciones de arreglos

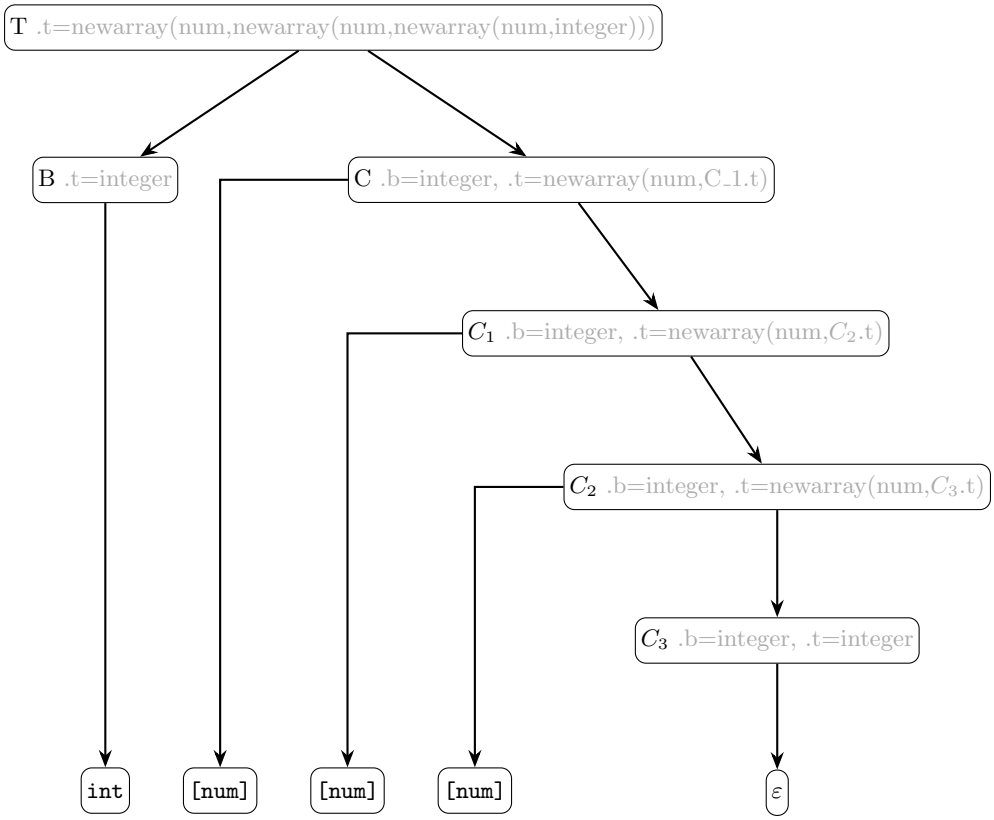
Cuadro 3.1: Reglas semánticas para declaraciones de arreglos

BNF	**Reglas semánticas**
T → BC	T.t = C.t; C.b = B.t
B → int	B.t = integer
B → float	B.t = float
C → [num]C_1	C.t = newarray(num, C_1.t); C_1.b = C.b
C → ε	C.t = C.b

Nota 3.1.1 Variables sintetizadas y heredadas

Cuando una variable semántica respeta el orden de evaluación de las reglas, se trata de una variable sintetizada: su valor depende de los valores devueltos por sus hijos. Si no respeta ese orden, es heredada: su valor depende de la regla padre. Una variable sintetizada corresponde siempre a un valor de retorno; una heredada se pasa como parámetro por referencia y puede modificarse en la regla hija.

3.1.1 Árbol sintáctico de prueba



3.1.2 Código fuente del analizador sintáctico

```
T(){
    return C(B();)
}
B(){
    if (input == "integer") {
        match("integer");
        return "integer";
    } else {
        match("float");
        return "float";
    }
}
C(b) {
    if(input == "[num]") {
        match("[num]");
        return newarray(num, C(b));
    } else {
        return b;
    }
}
```

3.2 Ejercicio 2 - Multiplicación de enteros de 0 a 9

Ejemplo 3.2.1 Ejemplo de cadena de entrada

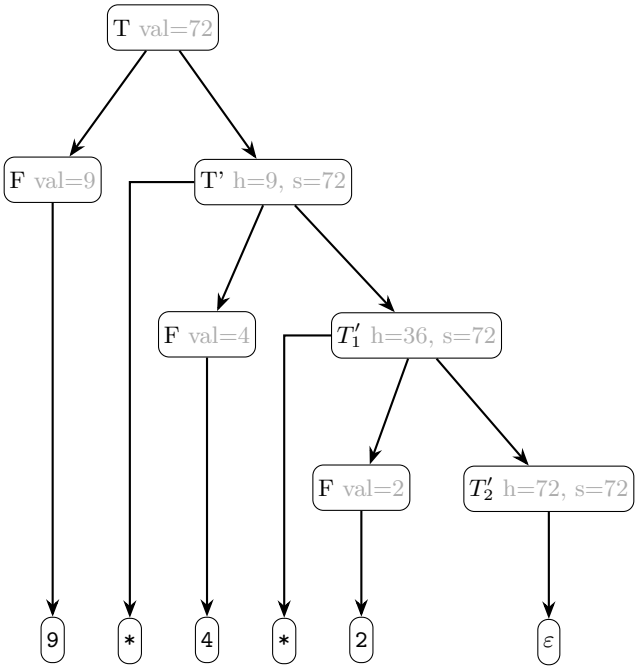
`9 * 4 * 2` es una cadena válida para la gramática y permite construir un árbol sintáctico cuya evaluación produce 72.

Cuadro 3.2: Reglas semánticas de multiplicación de enteros de 0 a 9

BNF	**Reglas semánticas**
$T \to FT'$	$T'.h = F.val; \quad T.val = T'.s$
$T' \to *FT'_1$	$T'_1.h = T'.h \times F.val; \quad T'.s = T'_1.s$
$T' \to \varepsilon$	$T'.s = T'.h$
$F \to 0$	$F.val = 0$
$F \to \ldots$	$F.val = \ldots$
$F \to 9$	$F.val = 9$

3.2.1 Árbol sintáctico del ejemplo

Para la cadena `9 * 4 * 2`, la evaluación de los atributos produce $T.val = 9 \times 4 \times 2 = 72$.



3.2.2 Código fuente del analizador sintáctico

El siguiente pseudocódigo implementa la gramática predictiva y conserva las reglas semánticas: h es el atributo heredado de T' y s su atributo sintetizado.

```
int T() {
    int f;

    f = F();
```

```
    return Tprima(f);       // T'.h = F.val
}

int Tprima(int h) {
    int f;

    if (preanalisis == "*") {
        match("*");
        f = F();
        return Tprima(h * f); // T_1'.h = T'.h * F.val
    }
    else {
        return h;             // T'.s = T'.h
    }
}

int F() {
    int valor;

    if (esDigito(preanalisis)) {
        valor = atoi(preanalisis);
        match(preanalisis);
        return valor;
    }
    else {
        error("Se esperaba un dígito entre 0 y 9");
        return 0;
    }
}
```

3.3 Ejercicio 3 - TDS de expresiones

Se considera la siguiente gramática para expresiones con suma, resta, paréntesis, identificadores y números:

$$E \to E + T \mid E - T \mid T,$$
$$T \to (E) \mid id \mid num.$$

El alfabeto de la gramática es

$$\Sigma = \{\texttt{num}, \texttt{id}, \texttt{(}, \texttt{)}, \texttt{-}, \texttt{+}\}.$$

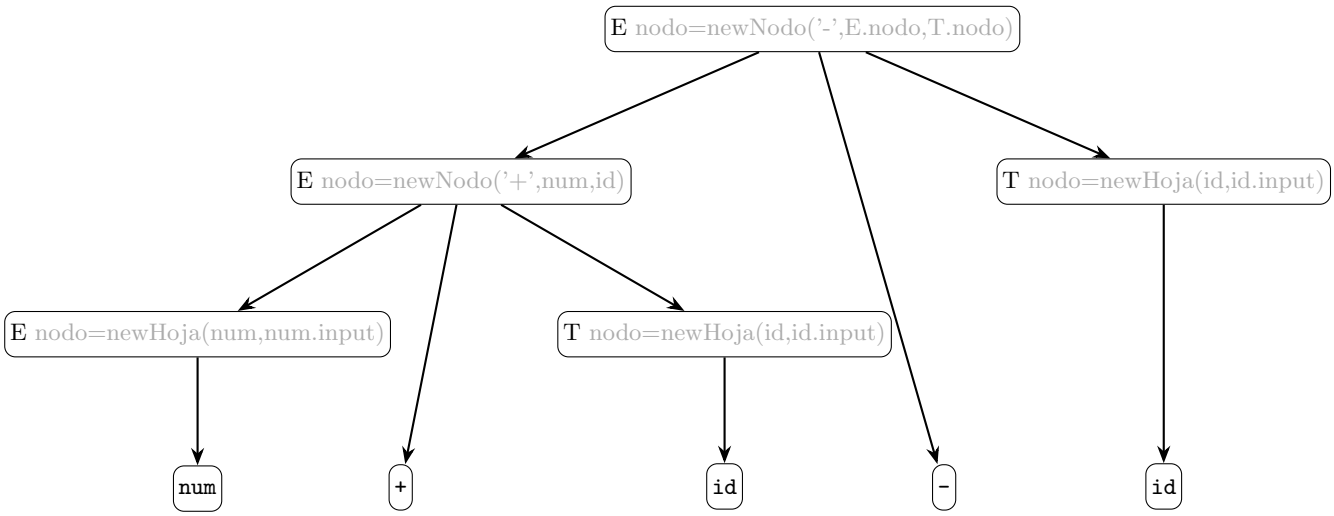
3.3.1 Reglas semánticas

Cuadro 3.3: Reglas semánticas para construir el AST

BNF	**Reglas semánticas**
E → E + T	E.nodo = newNodo('+', E.nodo, T.nodo)
E → E − T	E.nodo = newNodo('−', E.nodo, T.nodo)
E → T	E.nodo = T.nodo
T → (E)	T.nodo = E.nodo
T → id	T.nodo = newHoja(id, id.input)
T → num	T.nodo = newHoja(num, num.input)

3.3.2 Árbol sintáctico de prueba

Para la entrada `num + id - id`, la raíz del árbol representa la operación resta; su hijo izquierdo representa la suma. Por tanto, el AST resultante es `newNodo('-',newNodo('+',num,id),id)`.



3.3.3 Eliminación de la recursión por la izquierda

Definición 3.3.1 Reglas semánticas para la transformación predictiva

Para una producción con recursión por la izquierda de la forma $A \to A\alpha \mid \beta$, la transformación predictiva es

$$A \to \beta R,$$
$$R \to \alpha R \mid \varepsilon.$$

Las reglas semánticas asociadas se organizan así:

Cuadro 3.4: Transformación de $A \to A\alpha \mid \beta$

BNF	Reglas semánticas
$A \to \beta$	$R.i = f(\beta.x)$;
$\quad R$	$A.s = R.s$
$R \to \alpha$	$R_1.i = g(R.i, \alpha.y)$;
$\quad R_1$	$R.s = R_1.s$
$R \to \varepsilon$	$R.s = R.i$

En esta notación, el sufijo i significa que el atributo es heredado (*inherited*) y el sufijo s que es sintetizado. La variable del llamante es el atributo que se encuentra en el lado izquierdo de una acción: $A.s$ o $R.s$ pertenece al nodo que realiza la llamada, mientras que $R.i$ y $R_1.i$ son valores heredados que el padre transmite a sus hijos. La primera regla inicializa el acumulado; la segunda lo actualiza y lo propaga; la regla $R \to \varepsilon$ devuelve el acumulado sin modificarlo (se puede hacer analogía al caso base de una llamada recursiva).

En la transformación, β representa la alternativa no recursiva y α la parte que seguía al símbolo recursivo. Se mantienen separadas para distinguir el valor inicial de $R.i$ y el valor actualizado de $R_1.i$.

3.3.4 Conversión de la gramática de expresiones

Aplicando la transformación anterior a la gramática de expresiones, se obtiene la forma predictiva $E \to TR$ y $R \to +TR \mid -TR \mid \varepsilon$. Las acciones semánticas construyen el mismo AST que la gramática original:

Cuadro 3.5: Reglas semánticas para la gramática predictiva de expresiones

BNF	Reglas semánticas
$E \to T$	$R.i = T.nodo;$
$\quad R$	$E.nodo = R.s$
$R \to +T$	$R_1.i = newNodo('+', R.i, T.nodo);$
$\quad R_1$	$R.s = R_1.i$
$R \to -T$	$R_1.i = newNodo('-', R.i, T.nodo);$
$\quad R_1$	$R.s = R_1.i$
$R \to \varepsilon$	$R.s = R.i$

Nota 3.3.1 Atributos en la conversión

En $E \to TR$, $T.nodo$ es el valor sintetizado producido por el primer término y se entrega como atributo heredado $R.i$. En las producciones con + o −, $R_1.i$ es el nuevo acumulado heredado y $R.s$ es el resultado sintetizado que retorna al llamante. En el caso vacío, $R.s = R.i$; por eso $E.nodo = R.s$ conserva el árbol completo.

3.3.5 Conversión a codigo fuente

```
E() {
    return(R(T()));
}
R(i) {
    if (input == "+") {
        match("+");
        return R(newNodo('+',i,T()));
    } else if (input == "-") {
        match("-");
        return R(newNodo('-',i,T()));
    } else {
        return i;
    }
}
```

Nota 3.3.2 Reglas y acciones semánticas en el libro

El libro presenta estas construcciones como acciones semánticas (o esquemas de traducción). Funcionan con el mismo proceso de evaluación, pero se insertan como nodos adicionales del AST y se ejecutan en el orden requerido por la construcción de la traducción. En este capítulo las expresamos como reglas semánticas para hacer explícitos los atributos heredados y sintetizados.

3.4 Ejercicio 4 - Evaluación de expresiones con multiplicación

Se considera la gramática de expresiones con números, suma y multiplicación. El alfabeto es

$$\Sigma = \{n, +, *, (,), 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}.$$

Cuadro 3.6: Reglas semánticas para la evaluación de expresiones

BNF	Reglas semánticas
E → En	E.val = E.val
E → E_1 + T	E.val = E_1.val + T.val
E → T	E.val = T.val
T → T_1 * F	T.val = T_1.val * F.val
T → F	T.val = F.val
F → (E)	F.val = E.val
F → digito	F.val = digito.x
digito → 0	digito.x = 0
digito → ...	digito.x = ...
digito → 9	digito.x = 9

3.4.1 Conversión a gramática predictiva

//TODO

4

Clase 4 - Compiladores

4.1 Visión general del compilador



Un compilador divide la traducción en varias etapas especializadas. En este capítulo se estudian principalmente el análisis léxico y en parte el análisis sintáctico.

4.2 Análisis léxico

El analizador léxico lee las unidades básicas del lenguaje y tiene como objetivo reconocerlas como palabras (*lexemas*). Capta caracteres de la entrada hasta formar *lexemas* y también puede agruparlos según un criterio básico, formando familias lógicas codificadas (*tokens*), que luego entrega al analizador sintáctico.

4.2.1 Lexemas, tokens y patrones

- *Lexema*: secuencia de caracteres del programa fuente que coincide con un patrón; es una instancia de un token.
- *Token*: nombre que define una clase o tipo de patrón. Puede identificarse por su nombre o mediante un par ordenado formado por el nombre del token y su valor.
- *Patrón*: descripción, generalmente en lenguaje natural, de los lexemas que pertenecen a un token; es la definición del token.

Un *patrón* describe las condiciones que debe cumplir un *lexema* para ser reconocido como un *token*. Para evitar ambigüedades del lenguaje natural, los patrones se pueden definir mediante expresiones regulares (*Regular Expressions*, o *RegEx*).

Cuadro 4.1: Ejemplos de correspondencia entre lexemas, tokens y patrones

Lexema	Token	Patrón o descripción
`+, -`	Operador	Operadores de suma y resta
`2, 3, 5`	Dígito	Secuencia de dígitos
`if, while`	Palabra reservada	Palabras reservadas del lenguaje
`Índigo, Azul, Rosa`	Color	Nombre de un color
`Índigo, Azul, Rosa`	Nombre	Nombre de una persona
`while`	Variable	Una letra seguida de letras o dígitos

Nota 4.2.1 Ambigüedad léxica y error léxico

Existe ambigüedad léxica cuando no se puede determinar a qué token pertenece un lexema. Por ejemplo, `while` podría interpretarse como una palabra reservada o como el nombre de una variable. Ocurre un error léxico cuando no se encuentra ningún token al que pertenezca la palabra de entrada.

4.2.2 Expresiones regulares

Definición 4.2.1 Expresión regular

Una expresión regular es una notación para describir un conjunto de cadenas válidas mediante símbolos y operaciones bien definidas.

Casos base

- Si a es un símbolo terminal, su expresión regular es a.
- Si ε es una cadena vacía, su expresión regular es ε.

Operaciones

- **Unión:**
$$L \cup M = \{s \mid s \in L \text{ o } s \in M\} \quad \longrightarrow \quad r \mid s.$$
Si r y s son expresiones regulares, entonces $r \mid s$ es una expresión regular.
- **Concatenación:**
$$LM = \{st \mid s \in L,\ t \in M\} \quad \longrightarrow \quad rs.$$
Si r y s son expresiones regulares, entonces rs es una expresión regular.
- **Cerradura de Kleene:**
$$L^* = \bigcup_{i=0}^{\infty} L^i = L^0 \cup L^1 \cup L^2 \cup \cdots = \{\varepsilon\} \cup L \cup LL \cup \cdots .$$
Si r es una expresión regular, entonces r^* es una expresión regular.
- **Paréntesis:** se utilizan para agrupar expresiones regulares y definir el orden de las operaciones. Si r es una expresión regular, entonces (r) es una expresión regular.

Extensiones de expresiones regulares

Las extensiones usuales de las expresiones regulares incluyen:

- *Cerradura positiva de Kleene*: una o más instancias de r, representada por $r^+ = rr^*$.
- *Cero o una instancia*: representada por $r?$; equivale a $r \mid \varepsilon = \{r, \varepsilon\}$.
- *Clase de caracteres*: abreviación para una unión de símbolos, por ejemplo $(a \mid b \mid \cdots \mid z) = [a{-}z]$ o $[a : z]$.

Formas de definir patrones

Los patrones pueden definirse de dos maneras:

- **Por extensión:** se enumera una cantidad finita de casos, por ejemplo $\{+, -\}$.

- **Por comprensión:** se define una regla que describe una cantidad potencialmente infinita de casos, por ejemplo $\{0, 1, 2, 45, 99, \ldots\}$.

La cerradura de Kleene permite definir patrones por comprensión. Si $L = \{0, 1\}$, entonces

$$L^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, \ldots\}.$$

En el curso se utiliza la notación de potencia para indicar la cantidad de veces que se repite un patrón. Por ejemplo, si una variable se describe como $letra(letra \mid digito)^*$, una variable limitada a ocho caracteres puede expresarse como $letra(letra \mid digito)^7$.

Ejemplo 4.2.1 Ejemplos de expresiones regulares

1. Todas las cadenas y letras minúsculas que comiencen y terminen con a:

   `a([a-z]*a)?`

2. Cadenas formadas por a y b que no tengan tres b consecutivas:

   `a*b?(ab|ba)*b?a*`

3. Cadenas formadas por a y b que terminan en a, aa o ab:

   `(a|b)*a(a|b|`ε`)`

4. Cadenas formadas por a y b que no poseen un número impar de a consecutivas seguidas de un número impar de b consecutivas.

   Una solución propuesta es:

   $$(\texttt{aa}|\texttt{b}) * (\texttt{a}|\texttt{bb}*).$$

   La solución propuesta evita las cadenas que contienen un bloque impar de a seguido de un bloque impar de b, porque los bloques de a que aparecen antes de un bloque de b se construyen de dos en dos. Sin embargo, es incompleta y omite algunas cadenas válidas.

   Contraejemplos de omisión: las cadenas `aa` y `abb` cumplen el enunciado, pero la solución propuesta no las genera. En `aa` no aparece ningún bloque de b y en `abb` el bloque impar de a está seguido por un bloque par de b; por lo tanto, ninguna de las dos cadenas debe rechazarse.

   Una expresión regular que contempla esos casos y describe el lenguaje del enunciado es:

   $$\texttt{b} * ((\texttt{aa}) + \texttt{b} + |\texttt{a}(\texttt{aa}) * (\texttt{bb})+) * \texttt{a}*.$$

   El primer término permite un prefijo de b. Dentro del grupo repetido, $(aa) + b+$ representa un bloque par de a seguido de cualquier cantidad positiva de b, mientras que $a(aa)^*(bb)+$ representa un bloque impar de a seguido de un bloque par y positivo de b. El sufijo a^* permite terminar con cualquier cantidad de a sin crear un nuevo bloque seguido de b.

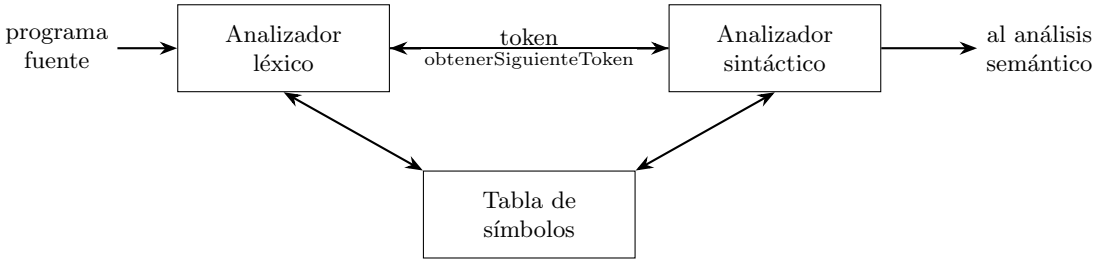
Consejo 4.2.1 Consejo del Profesor

Al crear una expresión regular, se recomienda buscar los casos que entran o no entran en el patrón y que podrían no cumplir las reglas deseadas del lenguaje natural traducido.

4.3 Análisis sintáctico

El analizador sintáctico hace énfasis en las frases y verifica que los tokens recibidos respeten la estructura definida por la gramática. Por ejemplo, una gramática sencilla para una frase puede escribirse como:

$$Frase \rightarrow Sujeto\ Verbo\ Predicado,$$
$$Sujeto \rightarrow Articulo\ Sustantivo,$$
$$Verbo \rightarrow \cdots .$$



Ejemplo 4.3.1 Interacción entre las etapas léxica y sintáctica

El analizador léxico recibe el programa fuente y entrega tokens al analizador sintáctico. Este puede solicitar el siguiente token mediante `obtenerSiguienteToken`. Ambos analizadores consultan y actualizan la tabla de símbolos cuando necesitan información asociada con los lexemas.

4.4 Relación entre las etapas

El compilador especializa las tareas en diferentes etapas, mientras que el traductor dirigido por sintaxis las unifica. Como el analizador sintáctico no puede reconocer palabras, no necesita conocer su significado: solo reconoce la estructura de las frases.

El analizador semántico sí puede necesitar el significado de las palabras. Por ello, el analizador sintáctico debe pasarle el índice correspondiente de la tabla de símbolos, de modo que pueda verificar el valor asociado. Un ejemplo de esta etapa es detectar un valor entero fuera del rango permitido.

Nota 4.4.1 Compilador y traductor dirigido por sintaxis

Si la definición BNF de un lenguaje contiene símbolos terminales a la derecha de sus producciones, se trata de un traductor dirigido por sintaxis, ya que el TDS realiza la comparación (*match*) de los caracteres. Si la BNF no contiene símbolos terminales a la derecha, puede utilizarse como base para un compilador.

4.5 Ejemplo de gramática y definiciones regulares

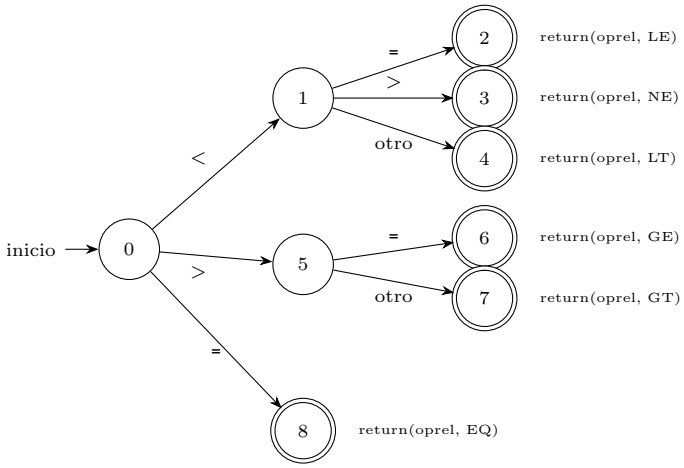
BNF

$$\begin{aligned}
instr &\rightarrow if\ expr\ then\ instr \\
&\mid if\ expr\ then\ instr\ else\ instr \\
&\mid \varepsilon \\
expr &\rightarrow term\ oprel\ term \\
&\mid term \\
term &\rightarrow id \mid num
\end{aligned}$$

Definiciones Regulares

$$\begin{aligned}
digito &\rightarrow [0-9] \\
digitos &\rightarrow digito^{*} \\
num &\rightarrow digitos(\,.digitos)?(E(\texttt{+} \mid \texttt{-})?digitos)? \\
letra &\rightarrow [A-Z\,a-z] \\
id &\rightarrow letra(letra \mid digito)^{*} \\
if &\rightarrow \texttt{if} \\
then &\rightarrow \texttt{then} \\
else &\rightarrow \texttt{else} \\
oprel &\rightarrow < \mid > \mid <= \mid >= \mid \texttt{=} \mid <>
\end{aligned}$$

oprel El siguiente autómata reconoce los operadores relacionales. Los estados circulares de doble borde son estados finales; al costado de cada uno se indica el token que debe retornarse.



Atributos de los tokens: `LE`: <= (menor o igual que), `NE`: <> (no igual), `LT`: < (menor que), `GE`: >= (mayor o igual que), `GT`: > (mayor que) y `EQ`: = (igual que).

Un diagrama de estados permite evaluar la pertenencia de la entrada a la salida. En este caso, cada recorrido desde el estado inicial hasta un estado final determina el operador relacional reconocido y el token que debe retornarse.

Código de `obtenerOpRel()` El siguiente fragmento muestra una implementación basada en el diagrama de transición de estados de *oprel*. Los estados 2, 3, 4, 6, 7 y 8 son terminales.

```
TOKEN obtenerOpRel()
{
    TOKEN tokenRet = new(OPREL);
    estado = 0;
    while (1) { /* repite el procesamiento de caracteres hasta que
        ocurre un retorno o un fallo */
        switch (estado) {
        case 0:
            c = sigCar();
            if (c == '<') estado = 1;
            else if (c == '>') estado = 5;
            else if (c == '=') estado = 8;
            else fallo(); /* el lexema no es un oprel */
            break;

        case 1:
            c = sigCar();
            if (c == '=') estado = 2;
            else if (c == '>') estado = 3;
            else estado = 4;
            break;

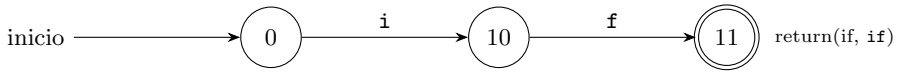
        case 2:
            tokenRet.atributo = LE;
            return(tokenRet);
        case 3:
            tokenRet.atributo = NE;
```

```
            return(tokenRet);
        case 4:
            retractar();
            tokenRet.atributo = LT;
            return(tokenRet);

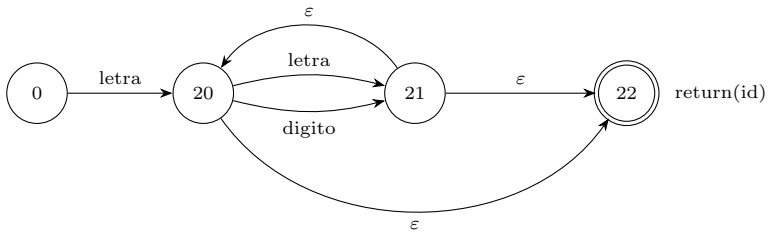
        case 5:
            c = sigCar();
            if (c == '=') estado = 6;
            else estado = 7;
            break;

        case 6:
            tokenRet.atributo = GE;
            return(tokenRet);
        case 7:
            retractar();
            tokenRet.atributo = GT;
            return(tokenRet);
        case 8:
            tokenRet.atributo = EQ;
            return(tokenRet);
        }
    }
}
```

if

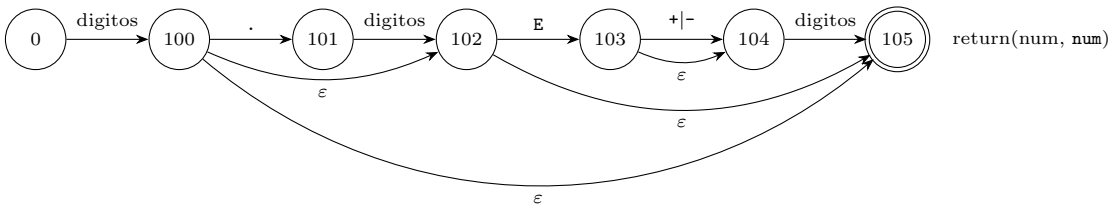


id Para $id = letra(letra \mid digito)^*$, el AFN utiliza una primera transición con *letra* y luego dos aristas paralelas desde el estado 20 al estado 21, una para *letra* y otra para *digito*. Las transiciones ε permiten terminar en cualquier momento o repetir la parte que comienza en 20.



El estado 22 es el único estado final. La transición $20 \xrightarrow{\varepsilon} 22$ permite terminar el identificador después de leer la primera letra o después de cualquier repetición. La transición $21 \xrightarrow{\varepsilon} 20$ permite repetir la lectura de una letra o un dígito.

num Para $num \longrightarrow digitos(.digitos)?(E(+ \mid -)?digitos)?$ se construye el siguiente AFN. El estado 105 es el único estado final.



Las transiciones ε representan las partes opcionales de la expresión: $100 \to 102$ omite la fracción, $100 \to 105$ omite toda la parte fraccionaria y el exponente, $102 \to 105$ omite el exponente, y $103 \to 104$ omite el signo opcional del exponente.

Nota 4.5.1 Aclaración sobre los AFN

Como trabajamos con autómatas finitos no deterministas (AFN), no importa si desde el estado inicial (nodo 0) podemos avanzar por varios caminos. Sí se produce una ambigüedad léxica si la misma entrada puede llegar a dos nodos terminales; por ejemplo, `if` puede llegar al nodo terminal 11 del AFN de la palabra reservada y al nodo terminal 22 del AFN de los identificadores.

4.5.2 Autómatas finitos no deterministas

Definición 4.5.1 Definición formal de un AFN

Un autómata finito no determinista se define mediante los siguientes elementos:

1. Un conjunto finito de estados S.
2. Un conjunto de símbolos de entrada Σ, que constituye el alfabeto de entrada. Se supone que ε, que representa la cadena vacía, nunca pertenece a Σ.
3. Una función de transición que proporciona, para cada estado y para cada símbolo de $\Sigma \cup \{\varepsilon\}$, un conjunto de estados siguientes:
$$\delta : S \times \big(\Sigma \cup \{\varepsilon\}\big) \longrightarrow \mathcal{P}(S).$$
4. Un estado $s_0 \in S$, distinguido como el estado inicial.
5. Un conjunto de estados $F \subseteq S$, cuyos elementos se distinguen como estados aceptantes o estados finales.

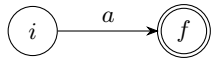
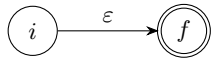
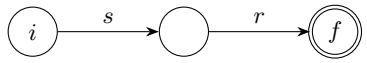
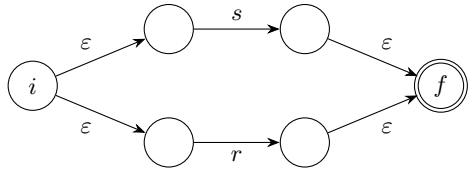
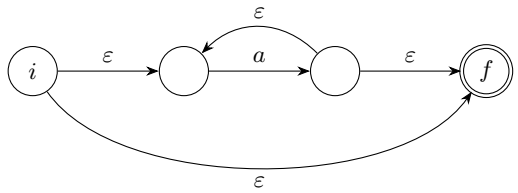
Un autómata finito no determinista (AFN) tiene las siguientes características:

- Puede tener transiciones con ε, que no consumen caracteres de la entrada.
- Puede tener transiciones indeterminadas: desde un mismo estado y con un mismo símbolo pueden existir varios estados siguientes.
- Puede representarse mediante una tabla de transición (la cual define la función de transición).

4.5.3 Construcción de Thompson

La construcción de Thompson permite formar un AFN a partir de una expresión regular de forma estandarizada y sencilla. Cada fragmento construido tiene un estado inicial y un estado final; estos fragmentos se combinan mediante transiciones ε.

Cuadro 4.2: Construcciones básicas de Thompson

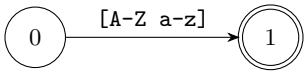
ER	Construcción AFN
a	
ε	
rs	
$s \mid r$	
a^*	

En la construcción de $s \mid r$, la entrada i se divide en dos caminos mediante ε y ambos caminos se unen en el mismo estado final. En la construcción de a^*, también se permite pasar directamente de i a f, porque la cadena vacía pertenece a a^*.

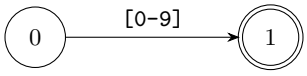
4.5.4 AFN de las definiciones regulares

A partir de las definiciones regulares se obtienen los siguientes fragmentos con la construcción de Thompson:

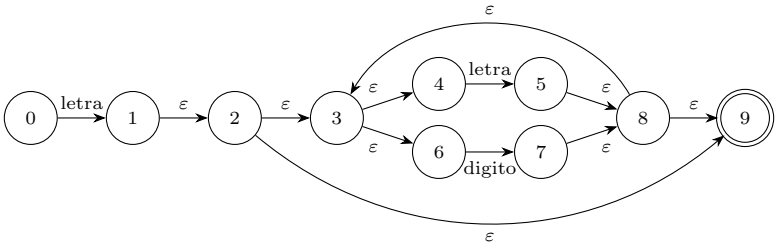
letra



digito



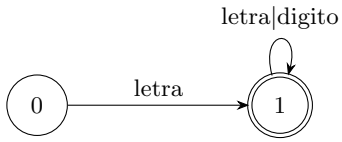
id Para $id = letra(letra \mid digito)^*$, primero se construye el fragmento de *letra* y luego se concatena con la construcción de Kleene aplicada a la unión $letra \mid digito$.



Las transiciones con ε permiten omitir la parte repetida, entrar a la unión, repetirla o salir de ella.
El estado inicial es 0 y el conjunto de estados finales es $F = \{9\}$.

Comparación con el AFD de id

El AFD equivalente elimina las transiciones ε y las decisiones indeterminadas. Para reconocer un identificador solo necesita recordar si ya se leyó la primera letra:



Definición 4.5.2 Autómata finito determinista (AFD)

Un autómata finito determinista es un autómata finito no determinista con las siguientes características:

a. No tiene transiciones definidas para el símbolo vacío ε.

b. Dado un estado i y un símbolo a del alfabeto, existe como máximo una transición desde i para a.

c. Para un estado i dado, existen n transiciones posibles para n símbolos diferentes del alfabeto.

Nota 4.5.2 Lenguajes reconocidos por AFN y AFD

Los AFN y los AFD pueden reconocer exactamente los mismos lenguajes. Esta clase de lenguajes se denomina lenguajes regulares y coincide con la clase de lenguajes que puede describirse mediante expresiones regulares. Por eso, una expresión regular puede convertirse en un AFN y luego en un AFD sin cambiar el lenguaje reconocido.

4.5.5 Algoritmo de Cerradura-ε (Algoritmo de Subconjuntos)

Nota 4.5.3 Conversión de AFN a AFD

Al convertir un AFN a un AFD, el AFD resultante no tiene transiciones ε ni transiciones indeterminadas. Escribir el código fuente de un AFN es más complicado que el de un AFD; por lo tanto, realizamos esta conversión. Si escribimos el código fuente de un AFD, ya tenemos el código de un analizador léxico.

Definición 4.5.3 Operaciones para la conversión de AFN a AFD

$$\begin{aligned} \text{cerradura}_\varepsilon(S_0) &= \{\text{conjunto de estados alcanzables por } \varepsilon \text{ desde } S_0\}\,, \\ \text{cerradura}_\varepsilon(T) &= \{\text{conjunto de estados alcanzables por } \varepsilon \text{ desde cada estado que pertenece a } T\}\,, \\ \text{mover}(T,a) &= \{\text{conjunto de estados alcanzables por } a \text{ desde algún } s \in T\}\,. \end{aligned}$$

El algoritmo de subconjuntos permite convertir un AFN en un AFD. A partir del estado inicial se calculan las cerraduras-ε y los movimientos posibles para cada símbolo de entrada.

```
c-eps(S_0) -> D_estados {
    while(hay un estado T sin marcar en D_estados) {
        marcar T;
        for (cada simbolo de entrada a) {
            U = c-eps(mover(T,a));
            if (U no esta en D_estados)
                agregar U como estado sin marcar en D_estados;
                D_transicion[T,a] = U;
        }
    }
}
```

Código del analizador léxico

El código de un analizador léxico basado en un AFD puede resumirse de la siguiente manera:

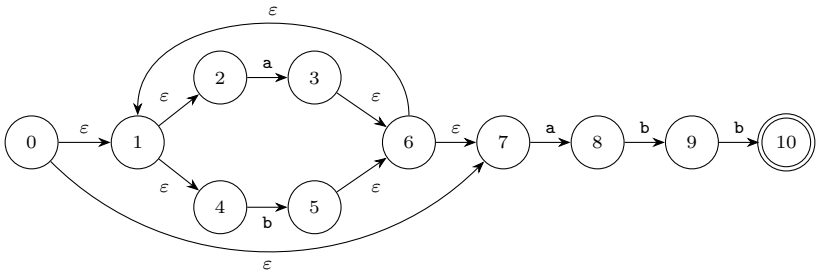
```
S = S_0;
c = siguienteInput();
while (c != EOF) {
    S = mover(S,c);
    c = siguienteInput();
}
if (S esta en F) return("si");
return("no");
```

Este codigo simula el comportamiento de un AFD, en un analizador léxico real, se reemplazaría el `return("si")` por la acción correspondiente, como agregar el lexema a la tabla de símbolos, y el `return("no")` por el reporte de un error léxico. En síntesis, el código del analizador léxico es el extracto anterior más la tabla de transiciones de estados.

Ejemplo de aplicación de algoritmo: AFN de Thompson

$$R = (a \mid b)^* abb \qquad \text{y} \qquad \Sigma = \{a, b\}.$$

El AFN obtenido mediante la construcción de Thompson es:



El estado inicial es 0, el estado final es 10, y las transiciones ε permiten repetir $(a \mid b)^*$ o continuar con abb.

Aplicación del algoritmo de subconjuntos

$$\begin{aligned}
& c_\varepsilon(S_0) = \{0,1,2,4,7\} = A. \\
\text{P/A:}\quad & U_a = c_\varepsilon(\text{mover}(A,a)) = c_\varepsilon(\{3,8\}) = \{3,8,6,7,1,2,4\} = \{1,2,3,4,6,7,8\} = B, \\
& U_b = c_\varepsilon(\text{mover}(A,b)) = c_\varepsilon(\{5\}) = \{5,6,7,1,2,4\} = \{1,2,4,5,6,7\} = C. \\
\text{P/B:}\quad & U_a = c_\varepsilon(\text{mover}(B,a)) = c_\varepsilon(\{3,8\}) = B, \\
& U_b = c_\varepsilon(\text{mover}(B,b)) = c_\varepsilon(\{5,9\}) = \{5,9,6,7,1,2,4\} = \{1,2,4,5,6,7,9\} = D. \\
\text{P/C:}\quad & U_a = c_\varepsilon(\text{mover}(C,a)) = c_\varepsilon(\{3,8\}) = B, \\
& U_b = c_\varepsilon(\text{mover}(C,b)) = c_\varepsilon(\{5\}) = C. \\
\text{P/D:}\quad & U_a = c_\varepsilon(\text{mover}(D,a)) = c_\varepsilon(\{3,8\}) = B, \\
& U_b = c_\varepsilon(\text{mover}(D,b)) = c_\varepsilon(\{5,10\}) = \{5,10,6,7,1,2,4\} = \{1,2,4,5,6,7,10\} = E. \\
\text{P/E:}\quad & U_a = c_\varepsilon(\text{mover}(E,a)) = c_\varepsilon(\{3,8\}) = B, \\
& U_b = c_\varepsilon(\text{mover}(E,b)) = c_\varepsilon(\{5\}) = C.
\end{aligned}$$

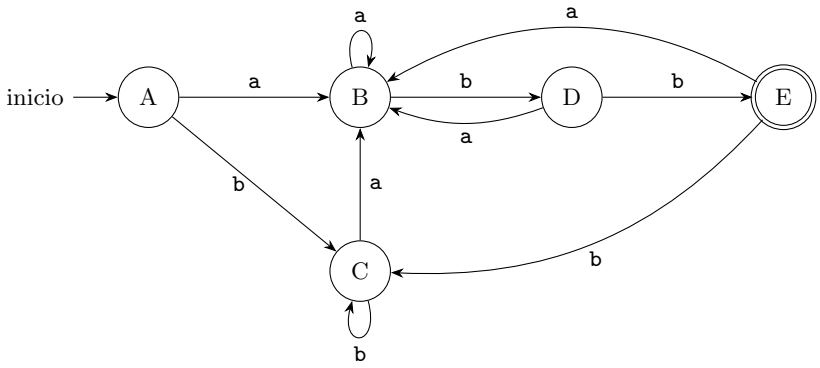
Por lo tanto,

$$D_{\text{estados}} = \{A, B, C, D, E\}.$$

Cuadro 4.3: Tabla de transiciones del AFD equivalente para el AFN de Thompson

Estado	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

El estado inicial del AFD es A y el conjunto de estados finales es $F = \{E\}$.



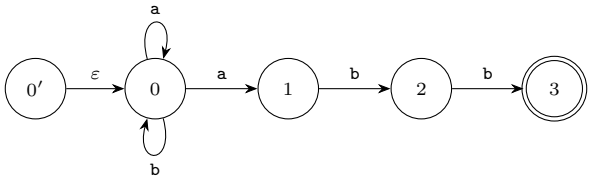
Ejemplo con AFN construido manualmente

Considérese el lenguaje descrito por

$$R = (a \mid b)^* abb, \qquad \Sigma = \{a, b\}.$$

Un AFN construido manualmente para este lenguaje puede representarse mediante las siguientes transiciones. Para mostrar primero el uso de la cerradura- ε, se agrega de manera didáctica un nuevo estado inicial $0'$ con una transición ε hacia el estado 0 original.

Variante didáctica con estado inicial $0'$



Como este AFN no tiene otras transiciones ε, la cerradura- ε solo es necesaria para incluir el estado 0 desde $0'$. Aplicando el algoritmo de subconjuntos:

$$c_\varepsilon(S_0) = c_\varepsilon(\{0'\}) = \{0', 0\} = A.$$

$$\begin{aligned}
\text{P/A:} \quad & U_a = c_\varepsilon(\text{mover}(A, a)) = c_\varepsilon(\{0, 1\}) = B, \\
& U_b = c_\varepsilon(\text{mover}(A, b)) = c_\varepsilon(\{0\}) = C. \\
\text{P/B:} \quad & U_a = c_\varepsilon(\text{mover}(B, a)) = c_\varepsilon(\{0, 1\}) = B, \\
& U_b = c_\varepsilon(\text{mover}(B, b)) = c_\varepsilon(\{0, 2\}) = D. \\
\text{P/C:} \quad & U_a = c_\varepsilon(\text{mover}(C, a)) = c_\varepsilon(\{0, 1\}) = B, \\
& U_b = c_\varepsilon(\text{mover}(C, b)) = c_\varepsilon(\{0\}) = C. \\
\text{P/D:} \quad & U_a = c_\varepsilon(\text{mover}(D, a)) = c_\varepsilon(\{0, 1\}) = B, \\
& U_b = c_\varepsilon(\text{mover}(D, b)) = c_\varepsilon(\{0, 3\}) = E. \\
\text{P/E:} \quad & U_a = c_\varepsilon(\text{mover}(E, a)) = c_\varepsilon(\{0, 1\}) = B, \\
& U_b = c_\varepsilon(\text{mover}(E, b)) = c_\varepsilon(\{0\}) = C.
\end{aligned}$$

Así,

$$D_{\text{estados}} = \{A, B, C, D, E\}, \qquad F_{\text{AFD}} = \{E\}.$$

El estado E es final porque contiene al estado final 3 del AFN.

Cuadro 4.4: Tabla de transiciones del AFD equivalente con estado inicial $0'$

Estado	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

Nota 4.5.4 Elección del estado inicial en el algoritmo de subconjuntos

El teorema del libro establece que el estado inicial del AFD se obtiene como la cerradura-ε del estado inicial del AFN:

$$S_{\text{inicial}}^{\text{AFD}} = c_\varepsilon(S_0).$$

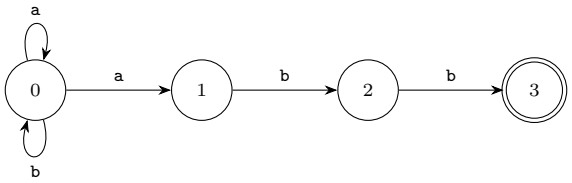
Cada estado del AFD representa un conjunto de estados del AFN. El algoritmo comienza con el conjunto $c_\varepsilon(S_0)$, antes de leer la entrada, y luego incorpora los conjuntos obtenidos mediante $c_\varepsilon(\text{mover}(T, a))$. Por ejemplo, en un AFN pueden aparecer $A = \{0\}$, $B = \{0, 1\}$ y $C = \{0, 2\}$; aunque los tres conjuntos contengan al estado 0, solo A se obtiene antes de procesar la entrada. Los conjuntos B y C se generan después de leer símbolos y representan estados alcanzados durante el recorrido.

En la construcción de Thompson, el estado inicial del AFN no tiene transiciones entrantes. Por consiguiente, ningún camino que comience en otro estado puede alcanzar S_0, ni mediante un símbolo de entrada ni mediante una transición ε. Así, en los AFN de Thompson, S_0 aparece únicamente en el subconjunto inicial $c_\varepsilon(S_0)$. En un AFN general, una transición que regrese al estado inicial puede hacer que S_0 aparezca en otros subconjuntos; estos siguen siendo estados generados después de leer la entrada. La regla del libro fija el único estado inicial del AFD antes de iniciar ese proceso.

La introducción de $0'$ es una variante didáctica para hacer explícita la cerradura-ε. En ese caso, $c_\varepsilon(\{0'\}) = \{0', 0\} = A$ y A es el conjunto inicial. Si se desea evitar la aparicion multiple del estado inicial en los subconjuntos, se puede agregar un nuevo estado $0'$ con transicion ε hacia el estado inicial (pero esto no es necesario).

Elección directa del estado inicial 0 El AFN original tiene un único estado inicial: 0. Las dos transiciones que salen de 0, con a y con b, representan la indeterminación propia del AFN; el estado inicial continúa siendo 0. Por el teorema:

$$c_\varepsilon(S_0) = c_\varepsilon(\{0\}) = \{0\} = A.$$



Como no hay transiciones ε, la cerradura no modifica los conjuntos. Aplicando el algoritmo de subconjuntos:

$$\begin{aligned}
& c_\varepsilon(S_0) = c_\varepsilon(\{0\}) = \{0\} = A. \\
\text{P/A:} \quad & U_a = \text{mover}(A, a) = \{0, 1\} = B, \\
& U_b = \text{mover}(A, b) = \{0\} = A. \\
\text{P/B:} \quad & U_a = \text{mover}(B, a) = \{0, 1\} = B, \\
& U_b = \text{mover}(B, b) = \{0, 2\} = C. \\
\text{P/C:} \quad & U_a = \text{mover}(C, a) = \{0, 1\} = B, \\
& U_b = \text{mover}(C, b) = \{0, 3\} = D. \\
\text{P/D:} \quad & U_a = \text{mover}(D, a) = \{0, 1\} = B, \\
& U_b = \text{mover}(D, b) = \{0\} = A.
\end{aligned}$$

Por lo tanto,

$$D_{\text{estados}} = \{A, B, C, D\}, \qquad F_{\text{AFD}} = \{D\}.$$

El estado D es final porque contiene al estado final 3 del AFN.

Cuadro 4.5: Tabla de transiciones del AFD equivalente con estado inicial 0

Estado	a	b
A	B	A
B	B	C
C	B	D
D	B	A

4.5.6 Resumen y Aclaraciones

Resumen 4.5.1 Analizador léxico

Para obtener el código fuente del analizador léxico:

1. Definir los patrones mediante expresiones regulares.
2. Convertirlos en un AFN mediante la construcción de Thompson.
3. Convertir el AFN en un AFD mediante el algoritmo de subconjuntos.
4. Teniendo la tabla de transición de estados, utilizarla como la función `mover()` en el código del analizador léxico. Se acepta el lexema si, al terminar la entrada, se llega a un estado final; de lo contrario, se rechaza (error léxico).

Nota 4.5.5 Aclaraciones sobre expresiones regulares y autómatas

- Una expresión regular **NO** valida la entrada; solo describe qué lexemas pueden pertenecer a un token.
- Un autómata es activo: valida la entrada al recorrerla y determina, mediante el estado final, si el lexema pertenece o no a algún token.
- En la definición de un lenguaje, todos los autómatas individuales (diagramas de estados para cada patrón de cada token) se conectan a un mismo estado inicial común. Así, la entrada se valida contra todos los patrones existentes para determinar a qué token pertenece o si es una entrada inválida (error léxico).

5
Ejercicios 1 - BNF

Pregunta 5.0.1 Ejercicios

Realizar el analisis de las siguientes gramaticas y eliminar la recursión por la izquierda, el factor común por la izquierda y el factor común indirecto por la izquierda. Luego generar el pseudocodigo para el analizador sintactico.

a) $S \to 0S1 \mid 01$
   $\Sigma = \{0, 1\}$

b) $S \to S(S)S \mid \epsilon$
   $\Sigma = \{(,)\}$

c) $S \to aSbS \mid bSaS \mid \epsilon$
   $\Sigma = \{a, b\}$

6
Ejercicio 2 - TDS

Pregunta 6.0.1 Ejercicio 2 - Movimiento de un robot

Escribir un TDS que permita dar una secuencia de instrucciones a un robot que se puede mover al este, oeste, norte o sur, y que retorne su posición final. Se propone la siguiente gramática:

$$SEC \to SEC\ INSRT \mid \texttt{inicio},$$
$$INSRT \to \texttt{este} \mid \texttt{oeste} \mid \texttt{norte} \mid \texttt{sur}.$$

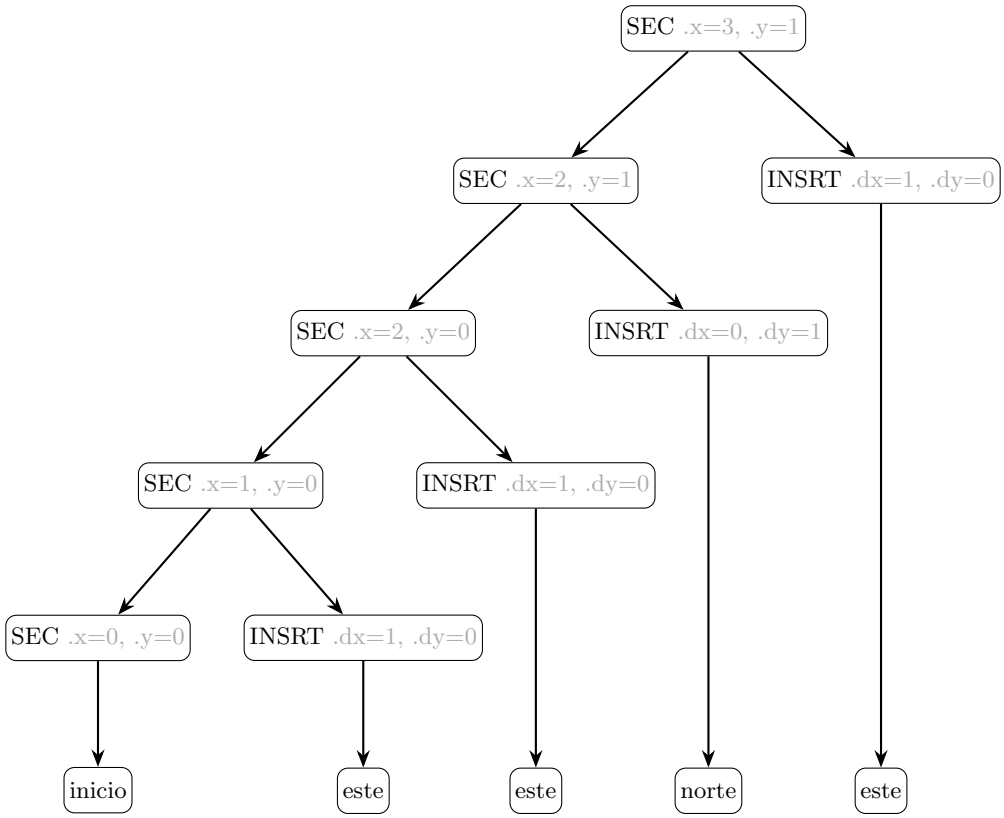
Ejemplo 6.0.1 Ejemplo de entrada

$$\texttt{inicio este norte} \quad \longrightarrow \quad (1, 1).$$

6.0.1 Solución con la gramática propuesta

Árbol sintáctico de la entrada

Se verificará el resultado para `inicio este este norte este`. Tomando el punto de partida como $(0, 0)$, cada instrucción modifica las coordenadas acumuladas. La cadena de prueba termina en $(3, 1)$.



Reglas semánticas iniciales

Por divide y vencerás, se asignan a SEC los atributos sintetizados x e y, y a $INSRT$ los desplazamientos dx y dy.

Cuadro 6.1: Reglas semánticas de la gramática propuesta

BNF	**Reglas semánticas**
SEC → SEC_1 INSRT	SEC.x = SEC_1.x + INSRT.dx; SEC.y = SEC_1.y + INSRT.dy
SEC → inicio	SEC.x = 0; SEC.y = 0
INSRT → este	INSRT.dx = 1; INSRT.dy = 0
INSRT → oeste	INSRT.dx = −1; INSRT.dy = 0
INSRT → norte	INSRT.dx = 0; INSRT.dy = 1
INSRT → sur	INSRT.dx = 0; INSRT.dy = −1

6.0.2 Conversión a una gramática predictiva

Eliminación de la recursión por la izquierda

La gramática no es predictiva porque SEC tiene recursión por la izquierda. Se aplica la transformación:

$$A \to A\alpha \mid \beta,$$
$$A \to \beta R,$$
$$R \to \alpha R \mid \varepsilon.$$

La gramática resultante es:

$$SEC \to \texttt{inicio}\ R,$$
$$R \to INSRT\ R \mid \varepsilon,$$
$$INSRT \to \texttt{este} \mid \texttt{oeste} \mid \texttt{norte} \mid \texttt{sur}.$$

No existe factor común por la izquierda.

Conjuntos Primero y predictividad

$$\begin{aligned} P(SEC) &= P(\texttt{inicio}) = \{\texttt{inicio}\}, \\ P(R) &= P(INSRT) \cup P(\varepsilon) \\ &= \{\texttt{este}, \texttt{oeste}, \texttt{norte}, \texttt{sur}, \varepsilon\}, \\ P(INSRT) &= P(\texttt{este}) \cup P(\texttt{oeste}) \cup P(\texttt{norte}) \cup P(\texttt{sur}) \\ &= \{\texttt{este}, \texttt{oeste}, \texttt{norte}, \texttt{sur}\}. \end{aligned}$$

Las alternativas comienzan con símbolos distintos; por ello no existen ambigüedades sintácticas ni semánticas y la gramática es predictiva.

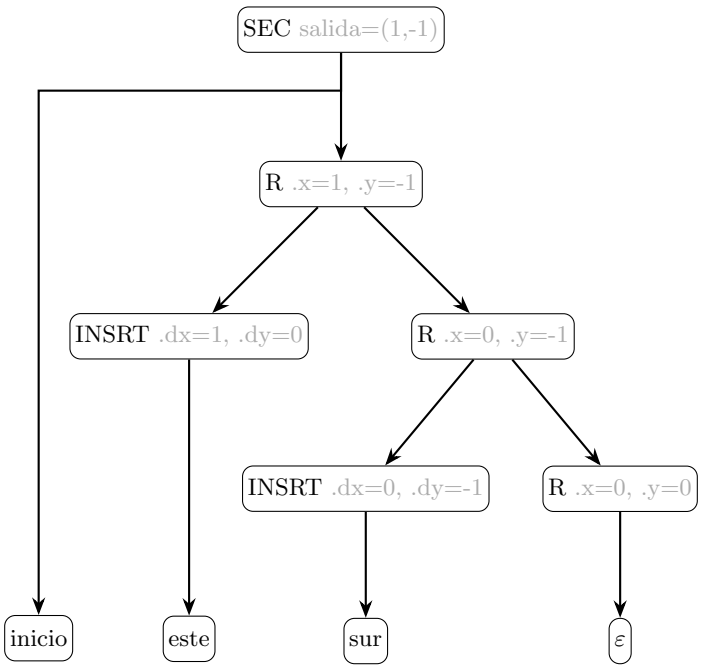
Reglas semánticas de la gramática predictiva

Cuadro 6.2: Reglas semánticas después de eliminar la recursión

BNF	**Reglas semánticas**
SEC → inicio R	`printf("(%d,%d)", R.x, R.y)`
R → INSRT R_1	R.x = R_1.x + INSRT.dx; R.y = R_1.y + INSRT.dy
R → ε	R.x = 0; R.y = 0
INSRT → este	INSRT.dx = 1; INSRT.dy = 0
INSRT → oeste	INSRT.dx = −1; INSRT.dy = 0
INSRT → norte	INSRT.dx = 0; INSRT.dy = 1
INSRT → sur	INSRT.dx = 0; INSRT.dy = −1

Árbol sintáctico con la gramática final

Para comprobar las nuevas reglas se utiliza `inicio este sur`, cuyo resultado es $(1, -1)$.



La evaluación empieza en el último R: el caso ε retorna $(0, 0)$; luego `sur` produce $(0, -1)$ y `este` produce $(1, -1)$.

6.0.3 Pseudocódigo del analizador sintáctico

Procedimiento para `SEC`

```
void SEC() {
```

```
    int x, y;
    match("inicio");
    R(&x, &y);
    printf("(%d,%d)", x, y);
}
```

Procedimiento para `R`

```
void R(int *x, int *y) {
    int dx, dy;
    if (preanalisis == "este"  || preanalisis == "oeste" ||
        preanalisis == "norte" || preanalisis == "sur") {
        INSRT(&dx, &dy);
        R(x, y);
        *x = dx + *x;
        *y = dy + *y;
    }
    else {
        *x = 0;
        *y = 0;
    }
}
```

Procedimiento para `INSRT`

```
void INSRT(int *dx, int *dy) {
    if (preanalisis == "este") {
        match("este");  *dx = 1;  *dy = 0;
    }
    else if (preanalisis == "oeste") {
        match("oeste"); *dx = -1; *dy = 0;
    }
    else if (preanalisis == "sur") {
        match("sur");   *dx = 0;  *dy = -1;
    }
    else {
        match("norte"); *dx = 0;  *dy = 1;
    }
}
```

Nota 6.0.1 Orden de evaluación

En $R \to INSRT\ R_1$, primero se reconoce la instrucción, después se evalúa recursivamente R_1 y, al retornar, se suman los desplazamientos. Los atributos x, y, dx y dy son sintetizados.

Nota 6.0.2 Variables sintetizadas y heredadas

Si una variable es pasada como parametro a una funcion es heredada, salvo el caso en que estemos pasando valores por referencia para utilizar multiples valores de retorno. En este ejercicio x y y son variables sintetizadas pasadas por referencia.

Todo list